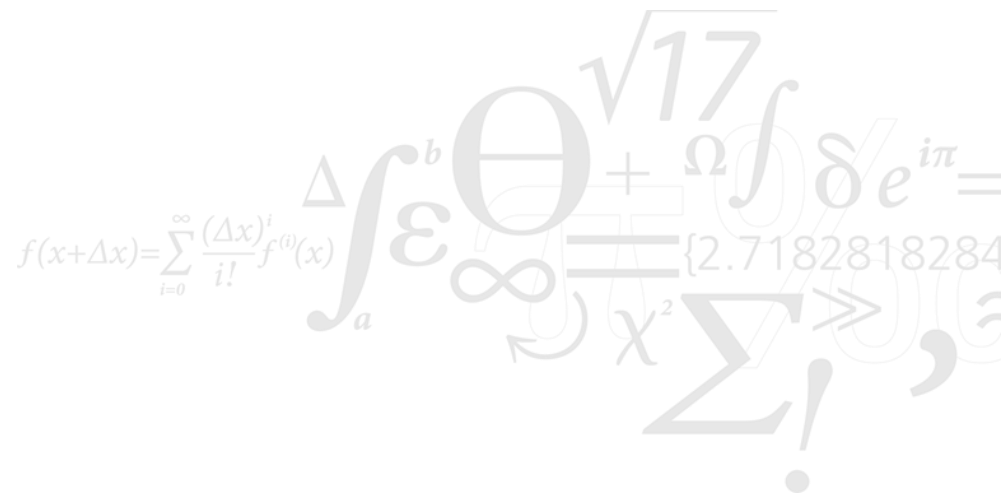


Encrypted Communication

- A protocol defines a sequence of steps involving several parties
- Characteristics of a protocol
 - Everyone knows the protocol in advance
 - Everyone agrees to follow the protocol
 - The protocol must be unambiguous
 - The protocol must be complete
- Typical “actors” in description of cryptographic protocols
 - Alice and Bob (parties who wish to communicate securely)
 - *Charlie and Dave are often added for multiparty protocols*
 - Eve (a malicious agent who wishes to eavesdrop on communication)
 - Mallory (a malicious attacker of any kind)
 - Trent (a trusted arbitrator – Trusted Third Party (TTP))

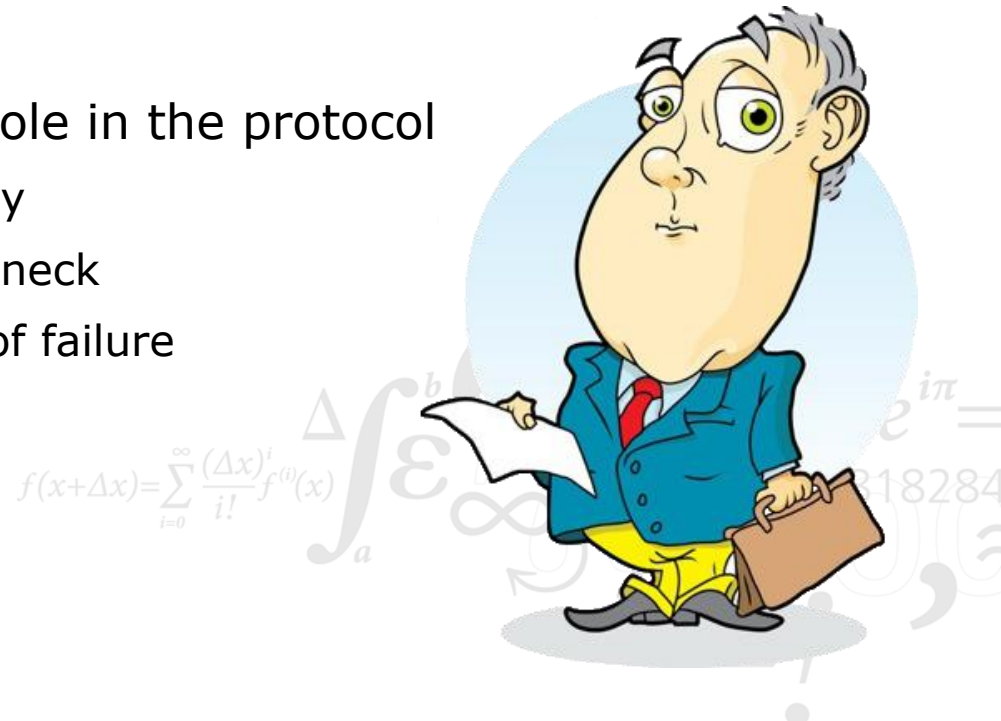
Cryptographic Protocols

- Cryptographic protocols
 - Describe how multiple parties employ cryptography together
 - *How each party uses the algorithms*
 - *How cryptographic keys are managed*
 - Key generation (not always covered in the protocol description)
 - Key distribution
- Three types of protocols
 - Arbitrated protocols
 - Adjudicated protocols
 - Self-enforcing protocols



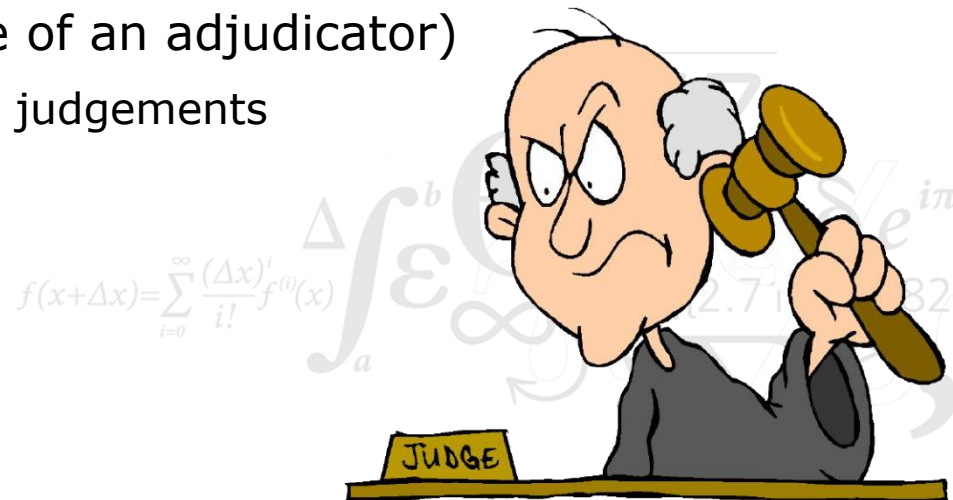
Arbitrated Protocols

- An arbitrator is an uninterested trusted third party
 - Examples: lawyers, notaries, banks
- All parties must trust the same arbitrator
- The arbitrator plays an active role in the protocol
 - the arbitrator introduces a delay
 - the arbitrator becomes a bottleneck
 - the arbitrator is a single point of failure



Adjudicated Protocols

- The costs of arbitration can be reduced by subdividing the protocol in two
 - 1) A non-arbitrated protocol to do transactions
 - 2) An arbitrated protocol to resolve disputes
- The non-arbitrated protocol must provide non-disputable evidence of the transaction (this is why we need “non-repudiation”)
- The adjudicator is only invoked in case of disputes (a judge in the legal system is a good example of an adjudicator)
 - All parties must agree to follow judgements

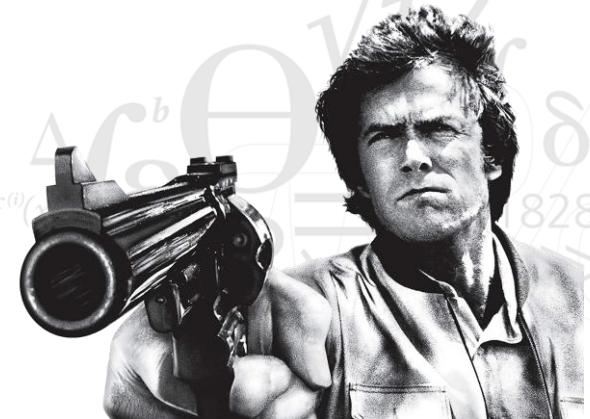


Self-Enforcing Protocols

- The protocol itself guarantees fairness
- It must be designed so that there cannot be any disputes
- Attempts to cheat must be detected before damage is done and the protocol stopped
- Self-enforcing protocols are preferred whenever possible
- Do you know examples of self-enforcing protocols?



The Enforcer

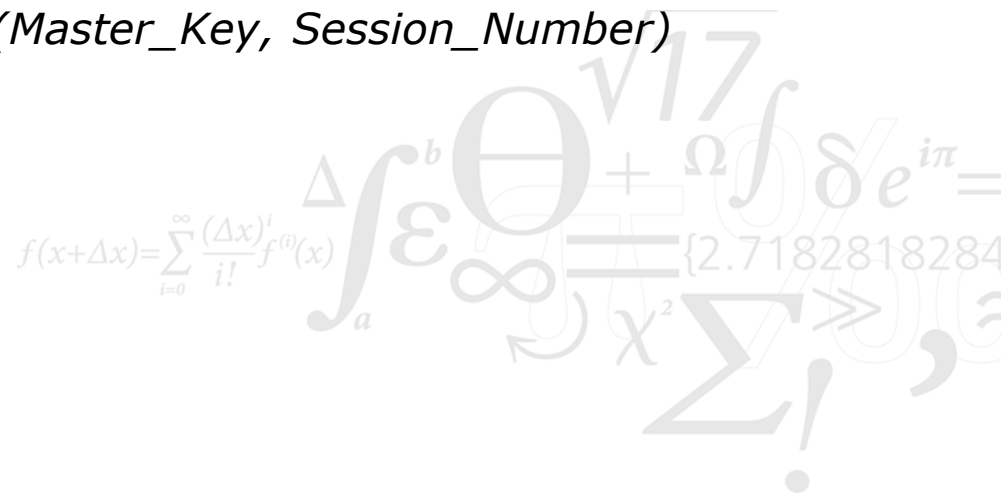


Encrypted Communication in Practice

- Handshake (taking the initial steps)
 - Agree on cryptographic protocol, including:
 - *Algorithms and Modes*
 - *Cryptographic session keys*
 - Authentication of Communicating Parties
 - *Single-side authentication (typically server)*
 - *Mutual authentication (both parties are authenticated)*
- Communication (steady state operation)
 - Send and receive messages securely
 - *Confidentiality protected through encryption*
 - *Integrity protected through MAC or digital signatures*
- Connection termination
 - Delete all session specific state (keys, cookies, ..)

Usage of Cryptographic Keys

- The following is a categorisation of cryptographic keys according to what they are used for:
 - **Data key**: Directly used for cryptographic purposes, e.g. encryption or authentication
 - **Key-encryption key**: Used to encrypt other keys, e.g. in key exchange or key storage
 - **Master key**: Used to generate other keys, e.g. in key derivation function (KDF).
Example: $\text{Session_Key} := \text{KDF}(\text{Master_Key}, \text{Session_Number})$



Cryptographic Key Management

- Key Generation
- Key Distribution
 - Key Exchange
 - *One party generates the key*
 - Requires key transport to all other parties
 - Key Agreement
 - *All parties influence the generation of the key*
 - Diffie-Hellman algorithm popular for two parties
- Key Storage
- Perfect Forward Secrecy (PFS)
 - Ensures that a session key derived from (a set of) long-term keys cannot be compromised if one of the long-term keys is compromised in the future

Key Generation

- Cryptographic keys must be intractable to guess
 - All possible cryptographic keys should be equally likely
- Typical key generation methods include:
 - Password Based Key Derivation Function
 - *generates a cryptographic key from a readable string*
 - Random number generator (RNG)
 - *Statistical RNG, not cryptographically secure*
Never use this for Cryptographic purposes!
 - *Pseudo-random number generator (PRNG)*
Must be seeded correctly, so use with care
 - *True random number generator*
Uses measurements of physical processes to generate "real" randomness - too expensive for most applications

True random numbers are available online: <https://www.random.org>

Key Exchange

Requirements

- In addition to being generated, the key must also be distributed to all legitimate parties
 - How to prevent others from seeing the key? **Encryption**
 - How to authenticate the legitimate parties (sender and receiver)? **ID verification, DS**
 - How to distributed the key to the legitimate parties? **Secure Key Exchange**
 - How to ensure that the key is fresh? **Policy- Key freshness, Nonce, Time Stamp**
 - How to verify that the legitimate parties received the key? **Mutual Auth, ACK**

If done remotely: Use cryptography (many different solutions)

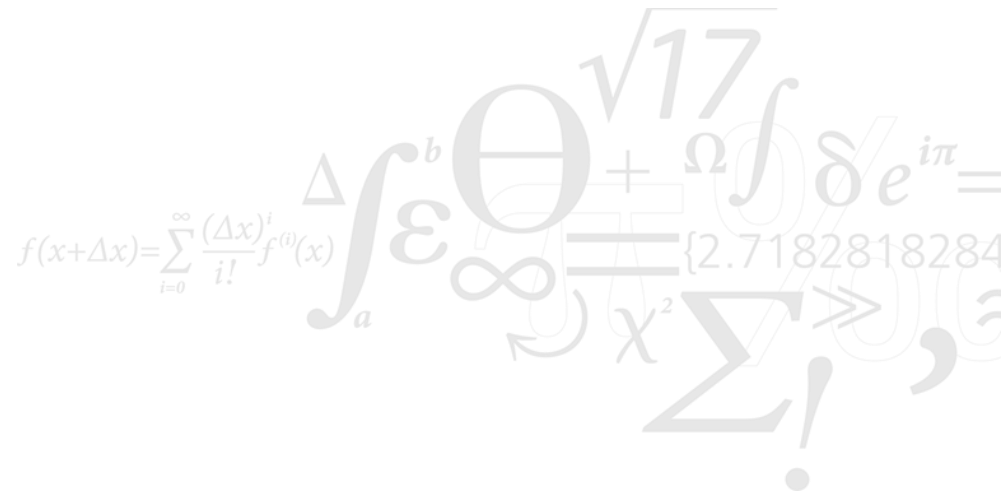
Sometimes easier: Personal key exchange

$$f(x+\Delta x) = \sum_{i=0}^{\infty} \frac{(\Delta x)^i}{i!} f^{(i)}(x)$$

Key Exchange

Techniques

- For key exchange we have the following options:
 - Generated by all parties working together (key agreement)
 - Generated by one party, then published in a public place
Public Key Infrastructure (PKI)
 - Generated by one party, then sent to the other (key transport)
 - Generated by a trusted third party and sent to all parties
(TTP key transport)
 - *TTP is often called Key Distribution Centre (KDC), e.g. used in Kerberos*



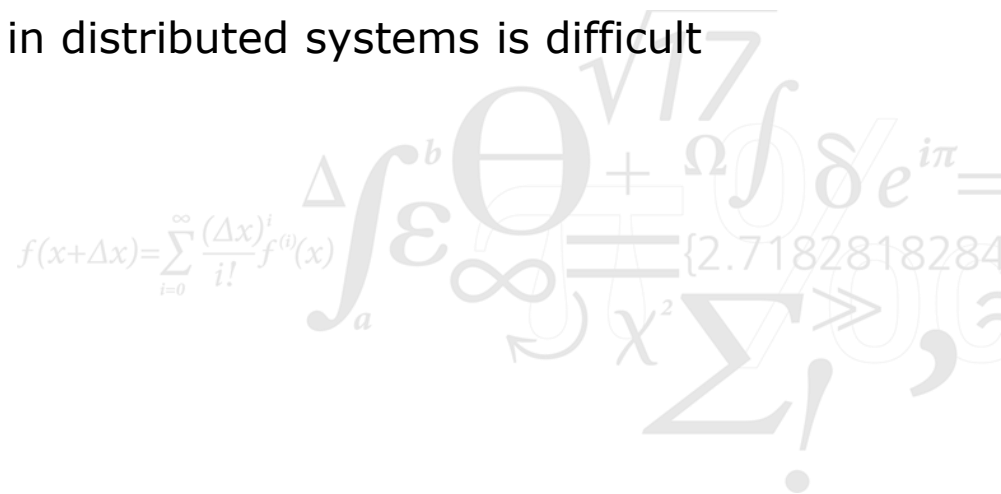
Key Storage

- Cryptographic keys must be stored *in use* and *at rest*
- In use
 - Keys must be available in memory when they are used
 - *Session keys (Data keys) are required when session is running*
 - *Other keys must only be available when needed*
 - Typically only during the handshake
 - Must be overwritten in memory immediately afterwards
 - Keys in memory was a problem with Heartbleed
- At rest
 - Keys and key generation material must be encrypted at rest
 - Example: Password Manager
- Hardware Secure Modules (HSM)
 - Dedicated hardware for key management and cryptographic operations



Key Expiration

- Keys can (in fact: should) expire sometime. Problems include:
 - How to keep track of key expiration? Centralized KMS → AWS KMS
 - How to inform all users of key expiration? Automated notification, rolling announcements
 - How to set up a new key? Secure Env, HSM
 - What happens after expiration?
 - *Archive old key material? How?*
 - *Delete old key material? How?*
 - Remember to delete all copies!
 - Verification of deletion in distributed systems is difficult



Key Compromise

- Worst case: Key has been compromised because
 1. An attacker has potentially had access to the key
 2. The corresponding cryptographic algorithm was broken
- What do we have to do?
 - Key must no longer be used in the future
 - *Key expiration (see above)*
 - All concerned parties have to be informed!
 - *Key Revocation*
 - Old data has to be protected
 - *Re-encryption? Re-signing?*
 - *Destruction of all copies of old data*

A company operates a private 5G network connecting industrial sensors, autonomous vehicles, and edge servers. All sensitive communication is encrypted using symmetric encryption. The security team stores cryptographic keys on individual devices. Over time, the company discovers that some devices have been compromised, and several employees who previously had access to the keys have left the organization.

The security manager proposes the following actions:

1. Generate unique encryption keys for each device.
2. Store all encryption keys in a centralized, access-controlled Key Management.
3. Rotate keys periodically and immediately revoke keys associated with compromised devices.
4. Give all network administrators access to every encryption key to simplify management.

Question:

Which approach provides the **most secure key-management strategy** for the company?

- A.** Store all keys locally on devices and allow administrators to share keys when needed.
- B.** Use unique keys per device, centrally manage them through an access-controlled KMS, rotate/revoke keys when necessary, and enforce least-privilege access.
- C.** Use one master encryption key for all devices and change it only when a device is compromised.
- D.** Give all administrators unrestricted access to all keys but change the keys every six months.

Effective cryptographic key management requires secure key storage, controlled access, key rotation, revocation, and lifecycle management.

Symmetric-Key Cryptography Protocol

- The following steps are required to setup communication using SKC
 - 1) Alice and Bob agree on a cryptosystem MIM, Weak Keys, Key Reuse
 - 2) Alice and Bob agree on a key Key interception
 - 3) Alice encrypts her plaintext with the key IV, weak padding, implementation bugs
 - 4) Alice sends the ciphertext to Bob Replay attack, DoS
 - 5) Bob decrypts the ciphertext with the key Error handling differences

How can these steps be compromised?

Problems of SKC

- Keys must be distributed in advance
- Keys must be kept secret from non-participants in the protocol
- A compromised key reveals all information encrypted with that key
 - Keys can be compromised:
 - *Weak keys (e.g. PRNG or KDF is weak) can be guessed*
 - *Man In The Middle (MITM)*
 - *Compromising hosts*
 - Probability of a host is compromised is p ,
probability of their shared key is compromised is $2p$
- Both parties have the same key
 - A compromised key can be used to masquerade as either party
 - *SKC cannot be used directly in adjudicated protocols*
- Different keys are needed for every pair of communicating parties
 - n users require $n(n-1)/2$ keys

Key-Exchange Protocols

- Assume a Key Distribution Center (KDC)
- Assume all participants share a key with KDC
- Shared-Key (symmetric) Protocols
 - Alice $\rightarrow$ KDC, request session key to talk with Bob
 - KDC generates Key, returns $K_A(K_{AB})$, $K_B(K_{AB})$ to Alice
 - Alice decrypts $K_A(K_{AB})$ and sends $K_B(K_{AB})$ to Bob
 - Bob decrypts $K_B(K_{AB})$, both now know K_{AB}
- Public-Key (asymmetric) Protocols
 - Alice gets Bob's public-key from KDC
 - Alice generates random session-key K and sends $K_{\text{pub}_B}(K)$ to Bob
 - Bob decrypts $K_{\text{priv}_B}(K)$, both now know K

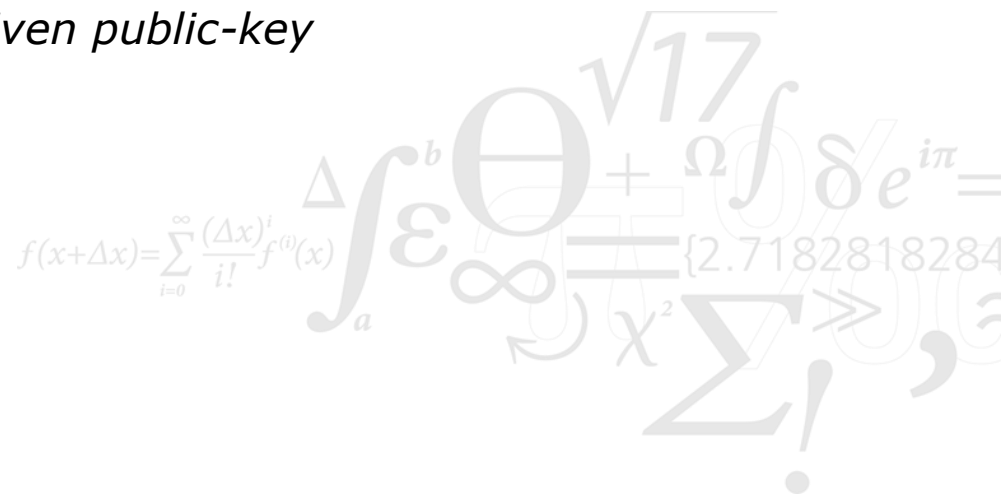
Public-Key Cryptography Protocol

- The following steps are required to setup communication with PKC
 - 1) Alice and Bob agree on cryptosystem **Weak Keys, Misconfiguration**
 - 2) Bob sends Alice his public key **CA compromise, Rouge certificates**
 - 3) Alice encrypts plaintext with Bobs public key **IV, Chosen Cipher text attacks**
 - 4) Alice sends ciphertext to Bob **Replay attack, DoS**
 - 5) Bob decrypts ciphertext using his secret key **Private Key compromise**

How can these steps be compromised?

Public-Key Distribution

- Public-key distribution must address:
 - Authenticity (Certification) of public-keys
 - *Linking public-keys to named identities*
 - Distribution of public-keys
 - *Obtaining somebody else's public-key*
 - *Distribute own public-key*
 - Revocation of public-keys
 - *Revoking published keys*
 - *Determining validity of a given public-key*

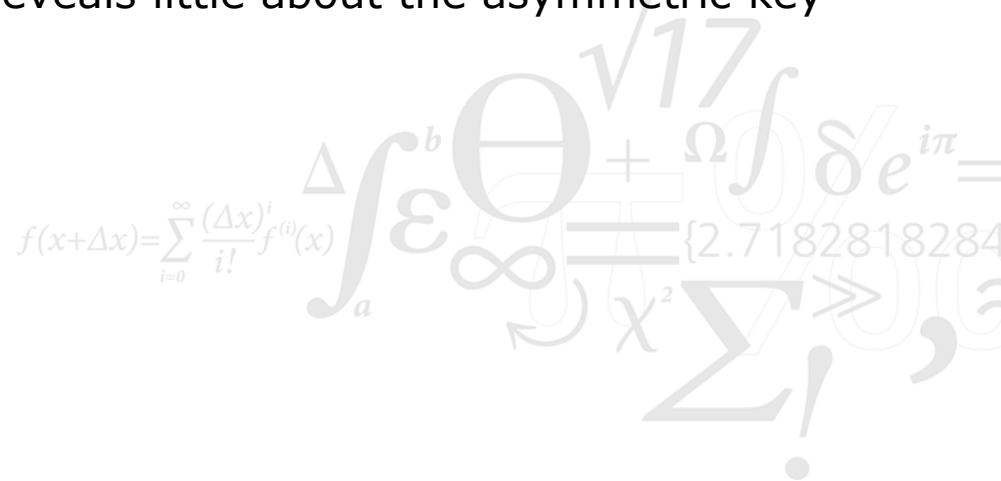


Hybrid Cryptosystems

- Asymmetric cryptography is 3 orders of a magnitude slower than symmetric
- Asymmetric cryptography is vulnerable to chosen plaintext attacks
- If $C = E(P)$, where P is plaintext with entropy n , a cryptanalyst can encrypt all possible plaintexts and compare with the cipher text (thus he learns the plaintext, but not the private-key)
 - The cryptosystem fails to achieve its goals without being broken
 - *Not enough to use strong algorithms and keys, must also use them in a meaningful way*

Hybrid Cryptosystems II

- Asymmetric cryptography can be used to encrypt a randomly generated symmetric key (session key)
- Advantages
 - Randomly chosen symmetric key has entropy close to the key-size
 - The symmetric key is short (compared to the encryption key)
 - *Encrypting symmetric key is shorter than encrypting message*
 - The encrypted symmetric key reveals little about the asymmetric key



Hybrid Cryptosystems III

- The following steps are required to setup communication with hybrid cryptography
 - 1) Bob sends Alice his public key
 - 2) Alice generates a random session key, K
 - 3) Alice encrypts K with Bobs public key
 - 4) Alice sends the encrypted key to Bob
 - 5) Bob decrypts K with his secret key
 - 6) Alice and Bob exchanges messages encrypted with the session key

*These steps still require authenticity of public-key
and authentication of end points*

Coffee Break



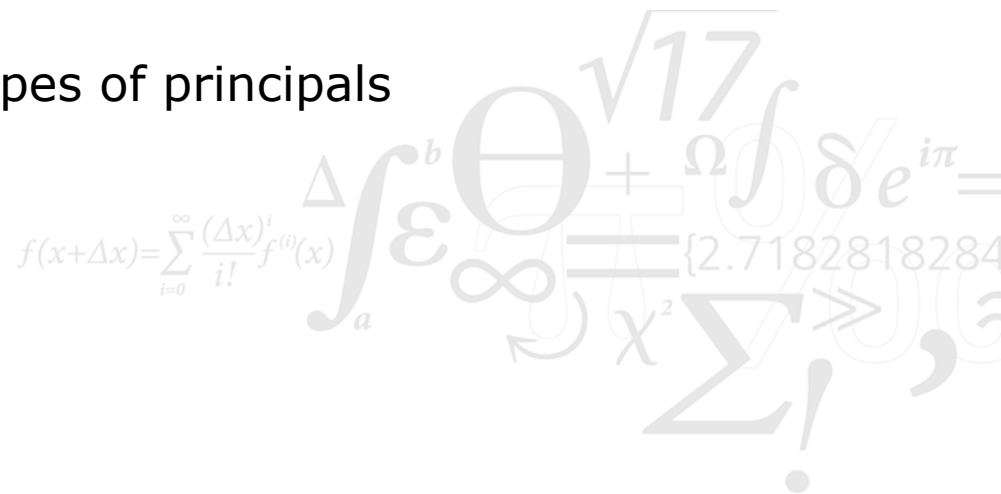
Public Key Distribution

- Public-key cryptography simplifies key distribution
 - secrecy of the encryption key is not required
- Authentication of Bob's public-key is required
 - In-Band (online)
 - *Public Key Infrastructures*
 - Key Distribution Centers (KDC)/Certificate Authorities (CA)
 - Out-of-Band
 - *Build into product*
 - *Published in Sunday Newspaper every week*

paper every week

Certification Authorities

- A certification authority (CA) guarantees that the key belongs to the named principal
- A principal can be:
 - A user
 - An attribute of the user (e.g., her role in within an organisation)
 - An organisation (e.g., a company or another CA)
 - A pseudonym
 - A piece of hardware/software
- Some CAs only allow certain types of principals



DTU

Homepage - 02270 Cybersecurity

learn.inside.dtu.dk/d2l/home/270894

Sci-Hub: removing... Python Dictionaries Python Strings sum() function in Py... Sci-Hub: removing... Phishing - Email He...

learn.inside.dtu.dk

Connection is secure

Cookies and site data

Site settings

About this page

Learn about its source and topic

security fundamentals, Fall 2025

Activities Content Assignments Discussions Video & Streaming Help

02270 Cybersecurity

Course Admin My Course

This course is open for students

When designing your course

Announcements

Welcome to Cybersecurity

Gaurav Choudhary posted on 01 Sep

Hi all,

Welcome to the 02270 Cybersecurity

Certificate Viewer: learn.inside.dtu.dk

General Details

Issued To

Common Name (CN) learn.inside.dtu.dk

Organization (O) <Not Part Of Certificate>

Organizational Unit (OU) <Not Part Of Certificate>

Issued By

Common Name (CN) Amazon RSA 2048 M02

Organization (O) Amazon

Organizational Unit (OU) <Not Part Of Certificate>

Validity Period

Issued On Tuesday, March 11, 2025 at 1:00:00 AM

Expires On Friday, April 10, 2026 at 1:59:59 AM

SHA-256 Fingerprints

Certificate 136dc5217c6b389c96b7450303c684b36626a34d45a3dac5d68e3114aa8f8891

Public Key c1e29ac2dabf907d71b454b4f09212383d684601adec981de9596e176bd72131



02270 Cybersec

Course Admin My Course

02270 Cyber

This course is open for stud
When designing your cours

Announcements

Welcome to Cybe

Gaurav Choudhary posted on 01 Sep

Hi all,

Welcome to the 02270 Cyb

Looking forward to the start of the course on Wednesday!

Certificate Viewer: learn.inside.dtu.dk

General

Details

Certificate Hierarchy

- ▼ Amazon Root CA 1
 - ▼ Amazon RSA 2048 M02
 - learn.inside.dtu.dk

Certificate Fields

- Serial Number
- Certificate Signature Algorithm
- Issuer
- ▶ Validity
- Subject
- ▶ Subject Public Key Info
- ▶ Extensions
- Certificate Signature Algorithm

Field Value

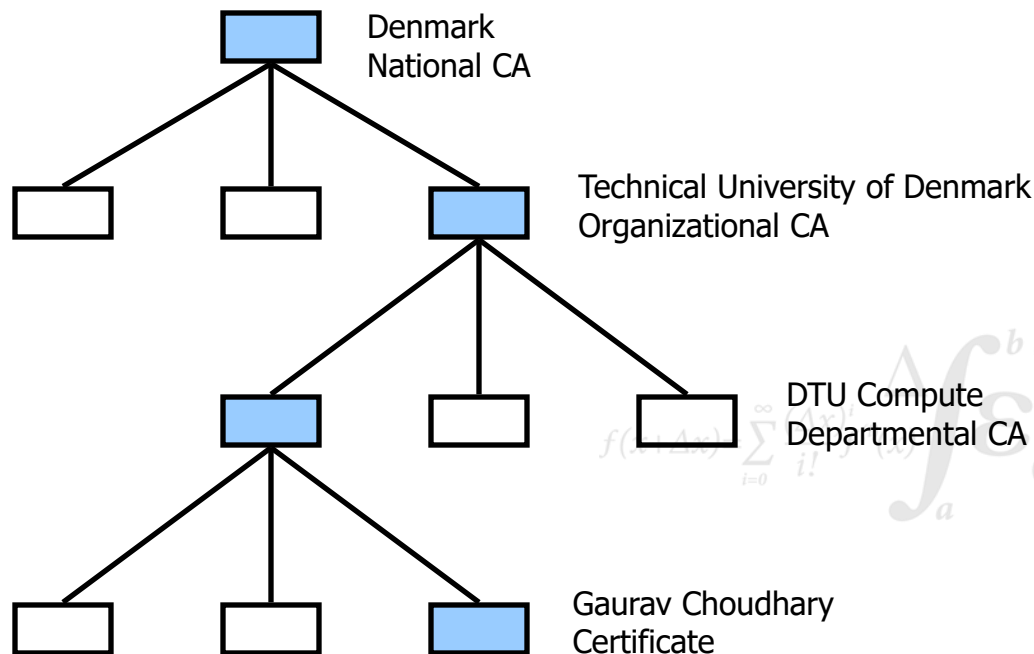
Export...

Obtaining a Certificate from a CA

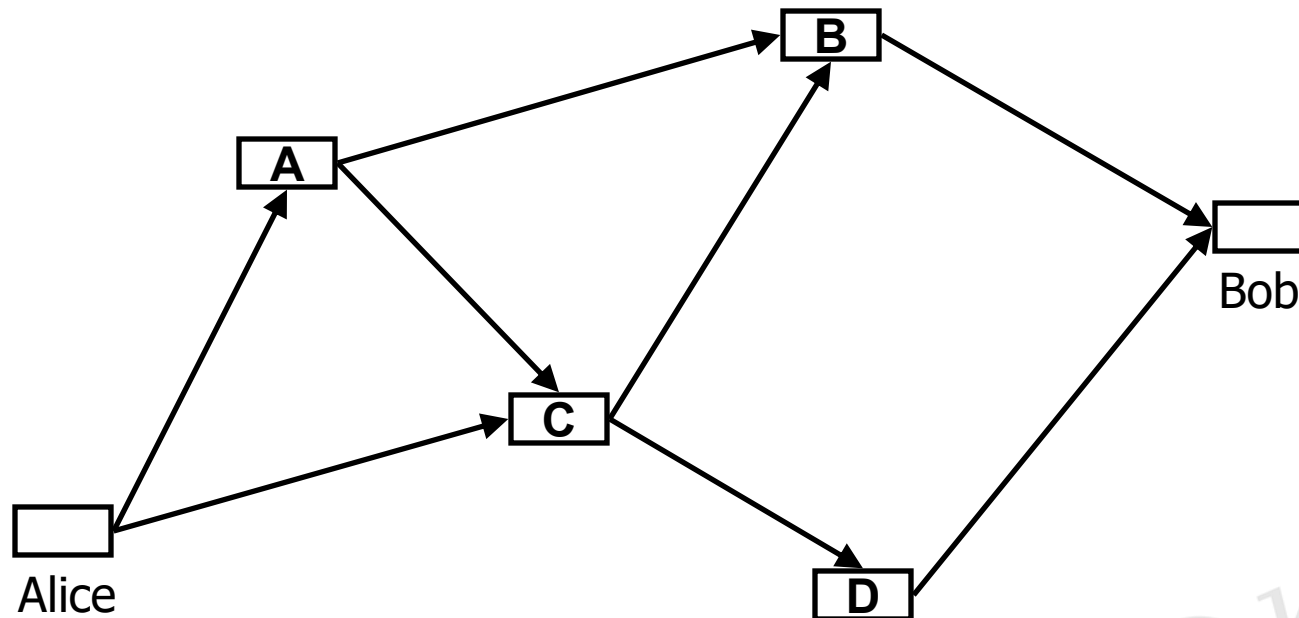
- Alice wishes to obtain a certificate from Charlie the CA
 1. Alice generates a public-/private-key pair and signs the public-key and identification information with the private-key
 - *Proves that Alice knows the private-key*
 - *Protects public-key and identification information in transit to CA*
 2. CA verifies Alice's signature on the public-key and her ID
 - *Verification may be done out-of-band*
 - Email/phone callback
 - Business/Credit bureau records, in-house records
 3. CA signs Alice's public-key and ID with the CA private-key
 - *Creating a certificate that binds Alice's public-key to her ID*
 4. Alice verifies the public-key, ID and CA's signature
 - *Proves that CA didn't substitute Alice's public-key*
 - *Protection of the certificate in transit from CA*
 5. Alice and/or CA publishes the certificate

PKI

- Certificate Authorities organized in a hierarchy
 - Only top-level CA's certificate needs to be known by everyone
 - Intermediate CAs have certificates signed by "superiors"
 - Certificate verification is done by verifying certificates from the principal towards the top-level CA (aka. "Root CA")



Alternatives Trust Hierarchies



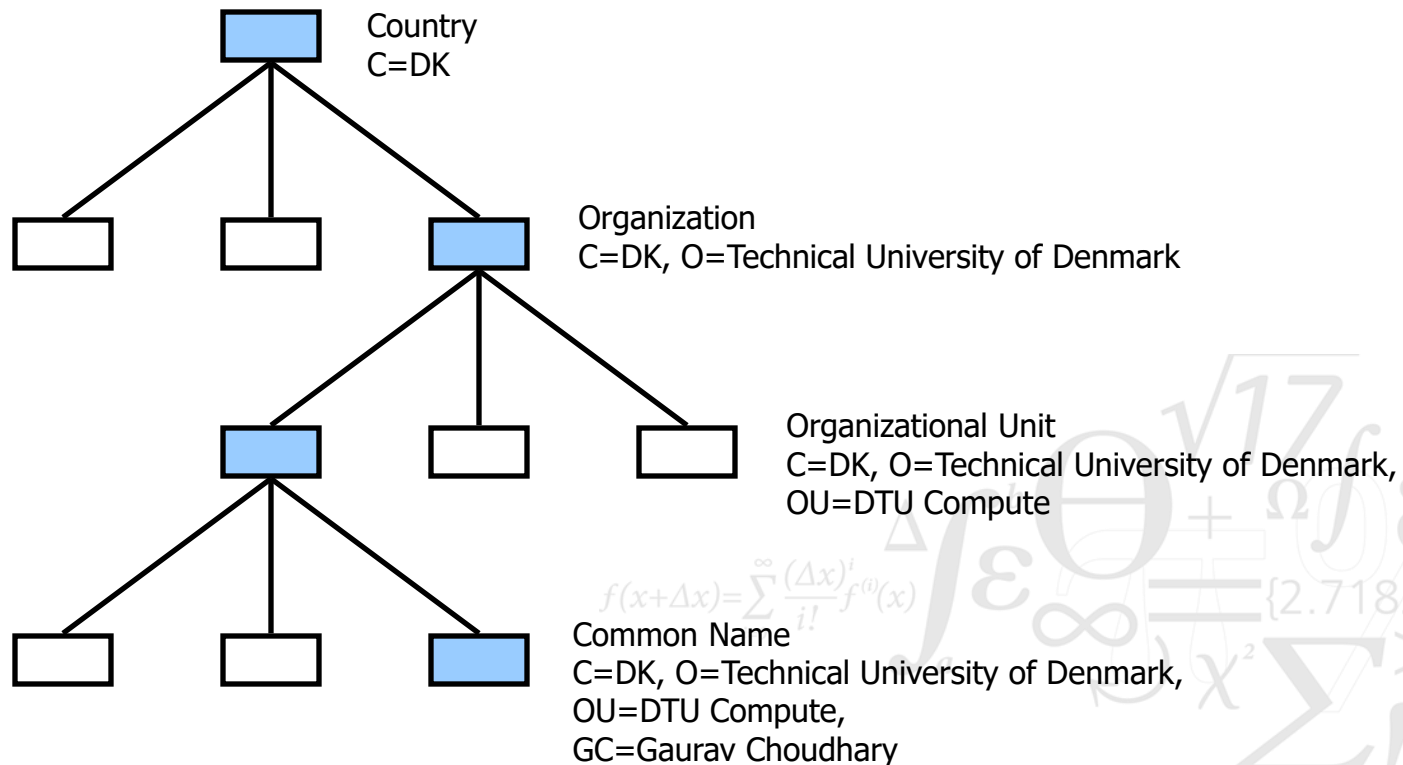
- Bob knows B and D who know A and C who know Alice $\Rightarrow$ Bob knows the key came from Alice
- Web of trust closer to human notions of trust
 - This is the method used in PGP (and GPG)

What's in a Name

- Alice uses a PKI to find Bob's certificate
 - Which PKI?
 - Which Bob?
- Names are context dependant
 - Bob is personally known by everyone in the village
 - *People have many names: Robert Johnson, Big Bob the sheriff, ...*
 - There are many people called Bob (or Robert) in the town
 - *People may not know that Robert Johnson = Big Bob*
 - The ASCII string "Bob" loses meaning in the big city
- Qualifiers can be added to names
 - Passport number
 - Central Person Register (CPR) number
- Are we looking for Bob123 or Bob421?

X.500 Naming

- X.500 defines Distinguished Names (DN) that uniquely names everything on earth



X.500 Distinguished Names

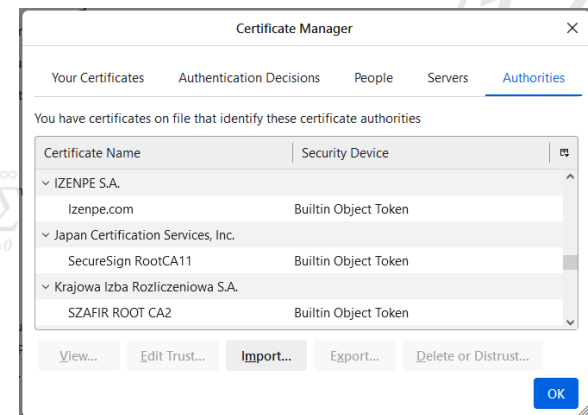
- Typical DN components
 - Country (C)
 - State or Province (SP)
 - Locality (L)
 - Organisation (O)
 - Organisational Unit (OU)
 - Common Name (CN)
- Problems with X.500 Distinguished Names
 - No rules for how the naming hierarchy should be organised
 - Hierarchical naming only works in clearly hierarchical contexts
 - *Governments, national telecoms providers, ...*
 - *Cannot accommodate nomadic people, stateless people, people with dual citizenship, ...*

What's in a Certificate

- A typical certificate contains
 - Public-key (e.g., 4096bit RSA key)
 - Identification (e.g., X.500 DN)
 - Validity
 - *Not valid before, not valid after*
 - TLS certificates are now valid for at most 13 months
 - Issuer identification
 - *Used to establish certification path back to root CA*
 - Issuer signature
- Extensions may qualify the certificate
 - Certificate can only be used for certain purposes
 - *Especially important for CA certificates*
 - Establish the domain of authority for the CA

Authority of a CA

- What is the root of authority for a CA?
 - TLS certificate binds a public-key to a business' web-server
 - *CA has no authority to register businesses*
 - *CA has no authority to register domain names*
- How do we get the CA root certificate?
 - Built into most browsers
 - Firefox comes with around 100 root-certificates pre-installed, incl.:
 - *Digicert*
 - *Entrust, Inc.*
 - *Google Trust Services LLC*
 - *Microsoft*
 - *Unizeto Sp. z o.o.*
 - *Verisign, Inc.*



Trust in PKI

- Root CA can compromise the security of everyone
 - Root CA must be infallible
 - No single authority in the world is trusted by everyone
- Trust in root CA is diluted by the certification path
 - Problem exposed by Uli Maurer's model
 - 90% in CA gives 60% trust in certificate with 5 levels of CAs
- Adding attributes magnifies this problem
 - Credential containing permissions to access a particular resource lends authority from a root CA who knows neither principal nor resource
 - Further down the credential chain permissions become more specific, but the authority of the root CA more diluted

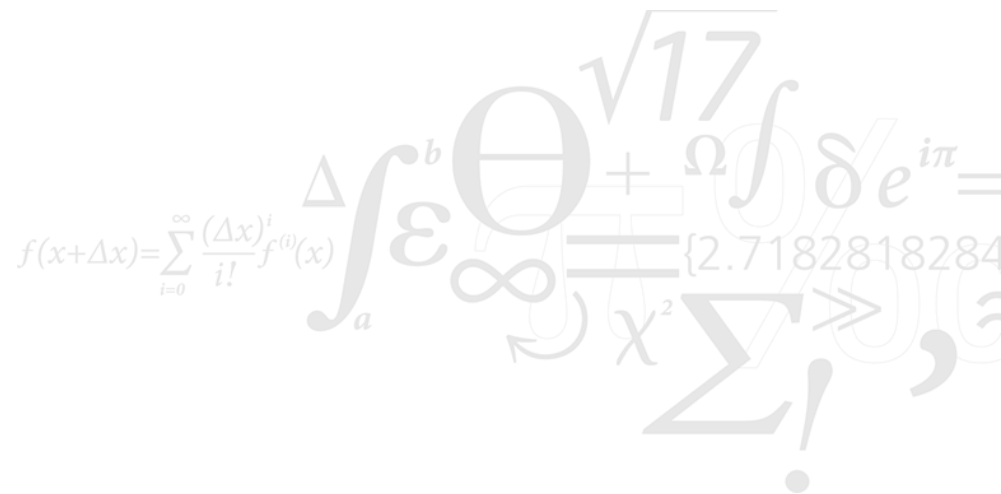


Certificate Revocation

- Certificates must be revoked whenever (*not if*) a private-key is compromised
- Revocation is measured by:
 - Speed of revocation: maximum delay from the compromise is discovered and the last use of the certificate?
 - Reliability of revocation: is it acceptable that someone sometimes may not have learned about the revocation?
 - Number of revocations: how many revocations can the system handle at the time?
- Revocation is either
 - Automatic, e.g., using short expiration times
 - Manual, e.g., using Certificate Revocation Lists (CRL)

Advanced Applications of Cryptography

- Asymmetric cryptography has many interesting applications
 - Blind signatures
 - Zero-Knowledge (ZK) protocols
 - Electronic voting systems
- We'll examine some of these in the following



Digital Signatures with RSA

- Some public-key crypto-systems allow either key to be used for encryption where the other key is used for decryption (e.g. RSA)

- The basic protocol is:

- 1) Alice encrypts the message with her private key, this effectively signs the message

$$\text{sign}(m) = m^d \pmod{n} = m' \pmod{n}$$

- 2) Alice sends the encrypted message to Bob

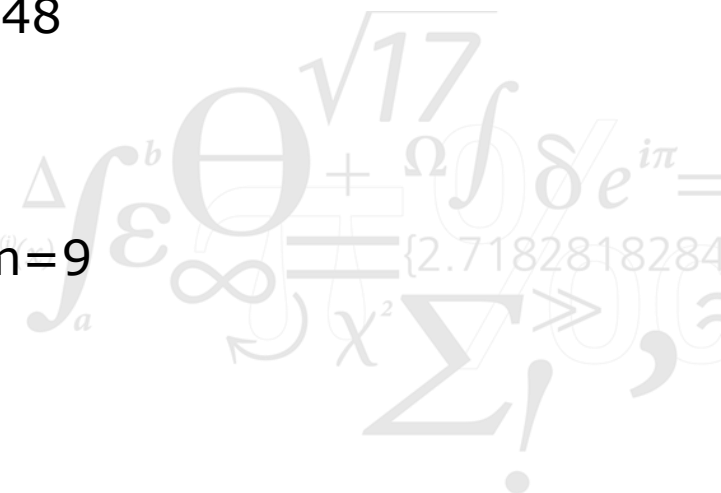
- 3) Bob decrypts the message with Alice's public key, this verifies Alice's signature

$$\text{verify}(m') = m'^e = (m^d)^e = m^{ed} = m \pmod{n}$$

- Asymmetric encryption is slow

- Normally sign a hash of the message instead of the message itself

- $p=11, q=13, n=p \cdot q = 143$
- $f(n)=(p-1)(q-1)=10 \cdot 12=120$
- $e=7$
- Compute $d \Rightarrow ed=1 \pmod{(p-1)(q-1)}$
- Solve: $7 \cdot d \equiv 1 \pmod{120}. \Rightarrow 103$
- Public key: $(e, n) = (7, 143)$
- Private key: $(d, n) = (103, 143)$
- $m=9$ (m as the hash of a longer message)
- Signature $s = m^d \pmod{n} = 9^{103} \pmod{143} = 48$
- Alice sends $(m, s) = (9, 48)$ to Bob.
- Verification = $s^e \pmod{n} = 48^7 \pmod{143} = m = 9$



Completely Blind Signatures

- Notaries don't always need to know what they sign
- Alice has a document (M) that she wishes Bob to sign using RSA
 - Bob should not learn the content of M
- Setup:
 - Bob has RSA keypair: public key (e, n) , private key (d, n) .
 - Alice has message M (numerical form, modulo n).

1) Alice picks a random "blinding factor" X such that $\gcd(X, n) = 1$.

$$M' = M \times X^e \pmod{n}$$

This hides M , because X^e looks random and only Alice knows X .

2) Alice sends the blinded document to Bob

3) Bob signs the blinded document

$$M'' = M' = (M')^d = (M \cdot X^e)^d \pmod{n}$$

4) Alice divides out the blinding factor (X), leaving the original document signed by Bob (M^d)

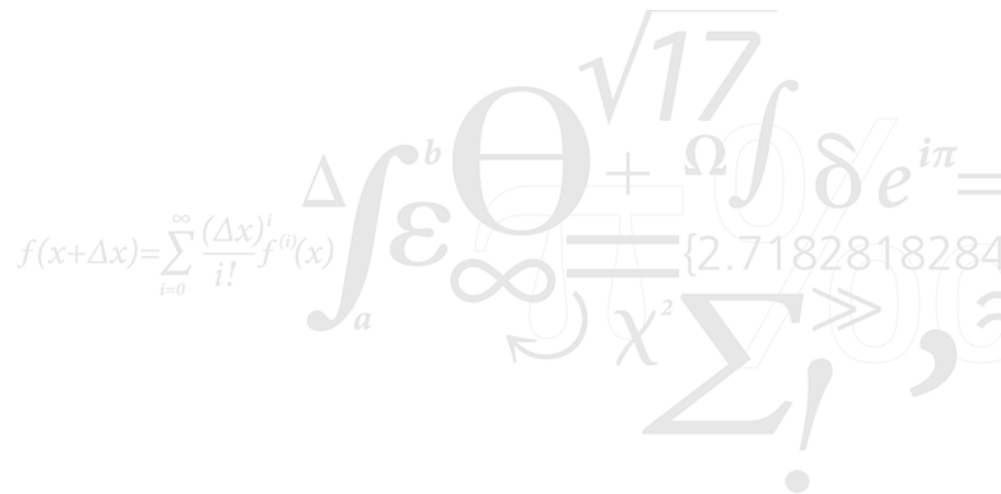
$$\sigma = M'' \cdot X^{-1} \pmod{n}$$

$$\sigma = M^d$$

This requires signature and multiplication to be commutative

Blind Signatures

- Completely blind signatures are of limited use
- If Bob **blindly signs** *anything*, he might end up signing something malicious (e.g., "I owe Alice \$1,000,000").
- Completely blind signatures → **no control at all** for the signer.
- That's why they are "of limited use."



Blind Signatures

- Bob uses partial verification:

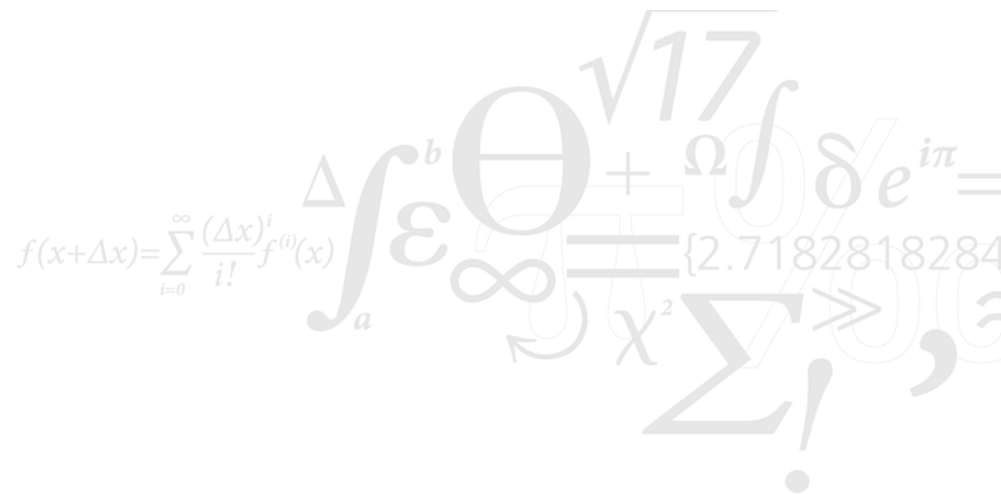
1. Alice has n documents $M_1, M_2, \dots, M_n$.
 - They are "synonymous" (all say the same thing, e.g., *"Alice has 1 e-coin"*).
 - Alice blinds each with a random factor X_i :

$$M'_i = M_i \cdot X_i^e \pmod{n}$$

2. Alice sends all n blinded documents $\{M'_1, \dots, M'_n\}$ to Bob.
3. Bob randomly chooses $n - 1$ **documents** and asks Alice to reveal their blinding factors.
 - For each revealed document, Bob unblinds and checks that it is a correct, honest message
 - If any look fraudulent, Bob refuses to sign.
4. Bob signs the **remaining one blinded message** (the one Alice did not reveal).
5. Alice unblinds the signed copy and obtains Bob's valid signature on one of her documents.

Blind Signatures

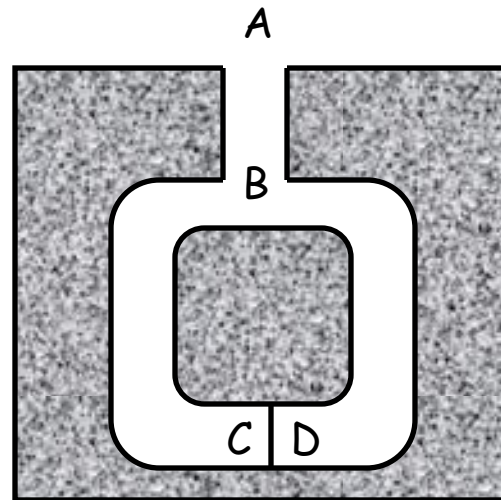
- Completely blind signatures = signer has no control → unsafe.
- Blind signatures with **cut-and-choose** give the signer **statistical assurance** that he isn't signing something malicious.
- Alice cannot cheat unless she guesses the one unchecked document, with only $1/n$ chance.



Zero Knowledge Proofs

the cave allegory

- How to prove that you know a secret without telling



How can Alice prove that she knows the secret password?

Alice goes into the cave, Bob moves to point B,
Bob calls what passage Alice should exit from,
Alice goes through the door (if necessary)
Alice exits from the correct passage

Zero Knowledge Proofs

basic protocol

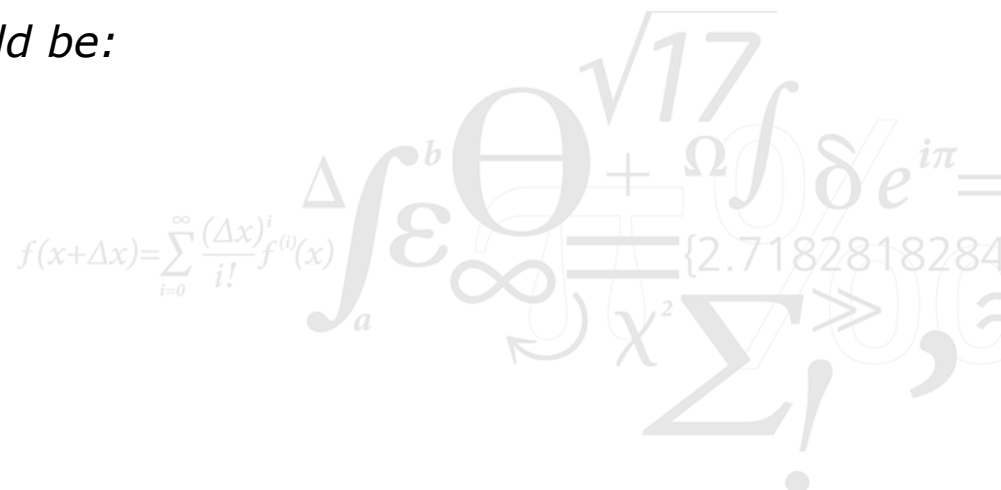
- Assume Alice knows the solution to a hard problem
 - 1) Alice use a random number to transform her problem into another hard problem, isomorphic to the first problem, she is able to solve the new problem using the solution to the original problem
 - 2) Alice commits to the new problem
 - 3) Alice reveals the new problem to Bob
 - 4) Bob asks Alice to :
 - a) prove that the two problems are isomorphic
 - b) reveal the solution (from 2)
 - 5) Alice complies to the request
 - 6) Alice and Bob repeats the protocol until Bob is satisfied Alice knows the secret
- This is essentially the “cut and choose” principle often used to divide a cake between two siblings

Secure Elections

- Requirements for elections on the Internet
 - 1) Only authorized voters can vote
 - 2) No one can vote more than once
 - 3) No one can determine for whom someone else voted
 - 4) No one can duplicate anyone else's vote
 - 5) No one can change anyone else's vote
 - 6) Every voter can make sure that his vote is counted

an additional requirement could be:

- 7) Everyone knows who voted



Voting with Blind Signatures

- 1) The **voter creates 10 sets of votes**, each set contains one vote for every possible outcome (e.g., two votes for a referendum) and a large random number
- 2) The voter **blinds all sets** of votes to the polling station
- 3) The polling station verifies that the voter hasn't voted before, requests the blinding factor for 9 of the 10 sets, opens and verifies the 9 sets and signs the 10th set of votes, returning it to the voter
- 4) The voter unblinds the votes and selects the appropriate vote
- 5) The voter encrypts his unblinded vote with the public key of the voting station
- 6) The voter sends the encrypted vote to the polling station
- 7) The polling station decrypts the vote, checks both the signature and that the serial number has not been received before (i.e., that only one vote from the set has been received) and registers the vote, the serial is made public so the voter can know that his vote is counted

Voting with Blind Signatures

1) Voter prepares votes

Alice (the voter) creates **10 sets** of votes.

Each set contains:

- A ballot for "YES"
- A ballot for "NO"
- A unique random serial number (to prevent double voting).

Example (just 1 set):

- YES vote: "YES | SN=12345"
- NO vote: "NO | SN=12345"

So 10 sets → 20 ballots total (YES/NO per set).

Voting with Blind Signatures

2) Voter blinds all ballots

Alice blinds all 10 sets with random blinding factors (so the polling station can't see content).

3) Polling station checks eligibility

- Station verifies Alice hasn't voted yet.
- Then it **challenges Alice**: "Reveal the blinding factors for 9 sets."
- Alice complies → station opens and checks 9 sets:
 - Verifies each set has both YES/NO and a valid serial.
 - Ensures there's no cheating (like "YES | YES").

Since 9 are correct, it's safe to assume the 10th is also correct.

- Station signs the 10th set **without unblinding it** and sends it back.

This is the **cut-and-choose guarantee**: probability Alice cheats successfully is 1/101/101/10.

Voting with Blind Signatures

4) Voter unblinds

Alice unblinds the signed 10th set.

Now she has an **authentic signed YES and signed NO vote**, both valid, but only she knows which she will use.

5) Voter casts her choice

Alice picks her actual choice — say "YES".

She encrypts "YES | SN=56789 | Signature" with the station's **public key**.

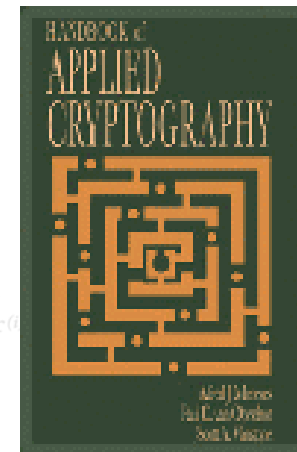
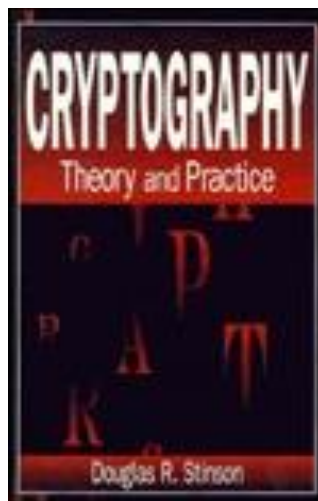
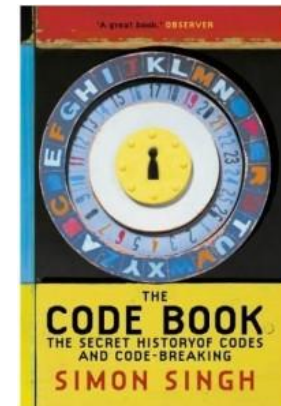
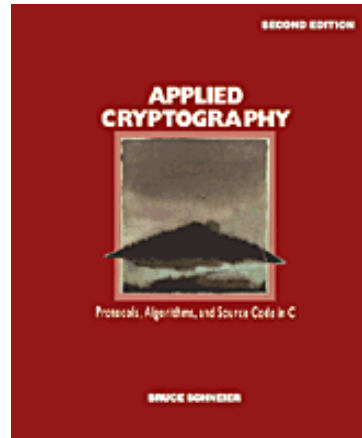
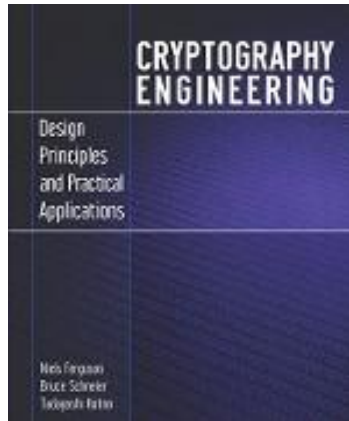
6) Voter submits

Alice sends the encrypted ballot to the polling station.

7) Polling station verifies & records

- Station decrypts with its private key.
- Verifies: The authority's own **signature** is valid and The **serial number** hasn't been used before (ensures only one vote per set).
- Registers the vote (e.g., "YES | SN=56789") in the tally.

To learn more



$$f(x+\Delta x) = \sum_{i=0}^{\infty} \frac{(\Delta x)^i}{i!} f^{(i)}(x)$$

<http://www.cacr.math.uwaterloo.ca/hac/>

